DEEP NEURAL NETWORKS — PROGRAMMING ASSIGNMENT 2

RNN, LSTM, GRU and Transformer | Experimental Report

Student: Korada Sriya Kushal Kalyan **BITS ID:** 2026AJ05601

1. Scope and experimental setup

This report summarizes the downloaded *DNN_Assignment_2-main* repository as supplied. Repository files were treated as read-only: no code, notebooks, figures or result files were changed. Problem 1 uses UCI HAR (128 steps × 9 channels; six classes) with subject-wise validation and training-only normalization, plus the synthetic adding task at T = 50, 100, 200 and 400. Three seeds are used for recurrent comparisons. Problem 2 implements attention and Transformer components from tensor operations, applies an encoder classifier to HAR, and trains a character-level causal language model on Tiny Shakespeare.

2. Implementation checks

The recurrent notebook records successful HAR classifier shape checks: input batch (4, 128, 9) produces six-class outputs. Parameter totals at hidden size 64 are 5,190 (RNN), 19,590 (LSTM) and 14,790 (GRU). The adding regressors return one scalar per sequence; data checks cover sequence dimensions, marker generation and subject-disjoint HAR splits. The notebook also records training-only standardization checks.

For scaled dot-product attention, every attention row sums to one and the custom output matches PyTorch’s reference with maximum absolute difference 5.96 × 10■■. The causal test assigns zero weight to future positions. The hand-computed three-token example agrees with code to displayed precision (maximum difference 5.51 × 10■■). For one encoder block (d_model 128, d_ff 512), attention projections contribute 66,048 parameters, feed-forward layers 131,712 and LayerNorm 512: total **198,272**, matching the notebook.

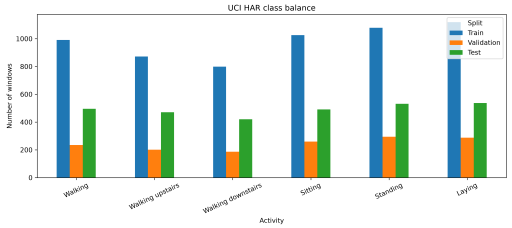
3. Problem 1 — HAR classification

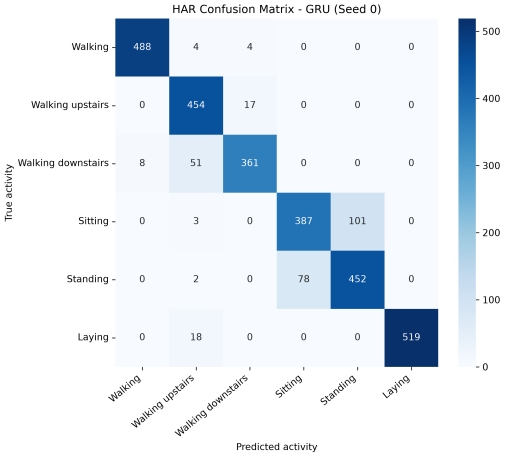
Table 1 reports three-seed mean ± sample standard deviation. GRU achieves the strongest mean accuracy and macro-F1; RNN is smaller and faster per epoch but substantially less accurate. GRU uses fewer parameters than LSTM, although its measured epoch time is higher in this environment.

Model	Test accuracy	Macro-F1	Parameters	s / epoch
RNN	74.65 ± 4.27%	73.65 ± 4.99%	5,190	3.53 ± 0.01
LSTM	88.37 ± 1.77%	88.06 ± 2.40%	19,590	4.41 ± 0.02
GRU	89.48 ± 0.73%	89.49 ± 0.70%	14,790	8.94 ± 0.19

Table 1. Three-seed HAR test metrics, parameters and epoch time.

GRU: 89.48% ± 0.73% accuracy and 89.49% ± 0.70% macro-F1. LSTM: 88.37% ± 1.77% and 88.06% ± 2.40%. RNN: 74.65% ± 4.27% and 73.65% ± 4.99%.





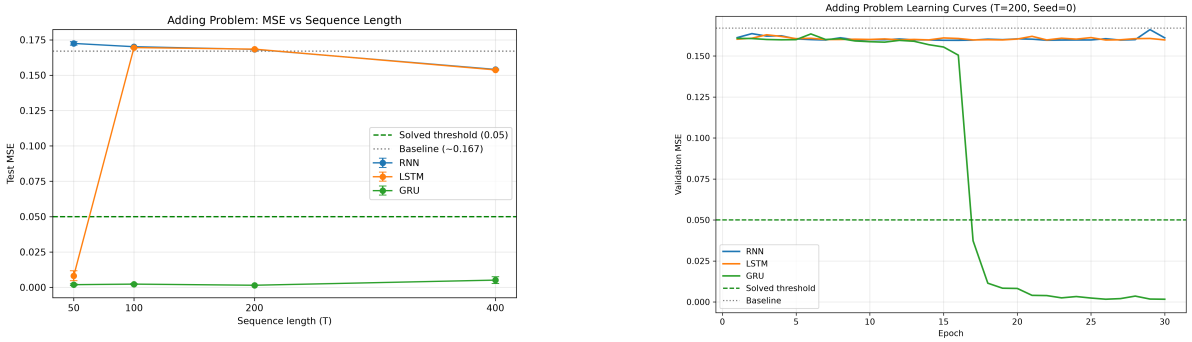
Figures 1–2. HAR class balance and confusion matrix for selected GRU run (seed 0; 90.30% accuracy).

4. Problem 1 — Adding task and gradient behaviour

A run is solved when test MSE < 0.05; the constant-prediction reference is approximately 0.167. GRU solves all tested sequence lengths, LSTM solves only T = 50, and RNN solves none. At T = 400, RNN and LSTM MSE fall below the reference baseline but remain above the required solved threshold.

Model	T=50	T=100	T=200	T=400
RNN	0.1724 ± 0.0013 0/3 solved	0.1701 ± 0.0002 0/3 solved	0.1682 ± 0.0002 0/3 solved	0.1540 ± 0.0002 0/3 solved
LSTM	0.0082 ± 0.0035 3/3 solved	0.1695 ± 0.0004 0/3 solved	0.1684 ± 0.0003 0/3 solved	0.1537 ± 0.0003 0/3 solved
GRU	0.0019 ± 0.0005 3/3 solved	0.0023 ± 0.0004 3/3 solved	0.0015 ± 0.0002 3/3 solved	0.0051 ± 0.0024 3/3 solved

Table 2. Test MSE mean ± SD and solved runs out of three at each sequence length.



Figures 3–4. MSE against sequence length and T = 200 learning curves.

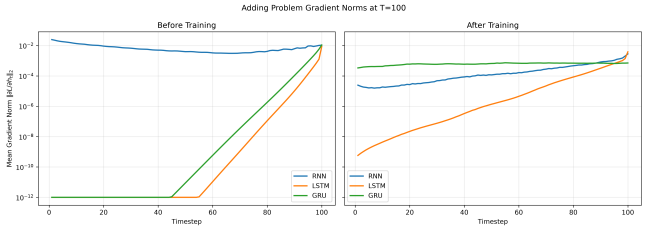
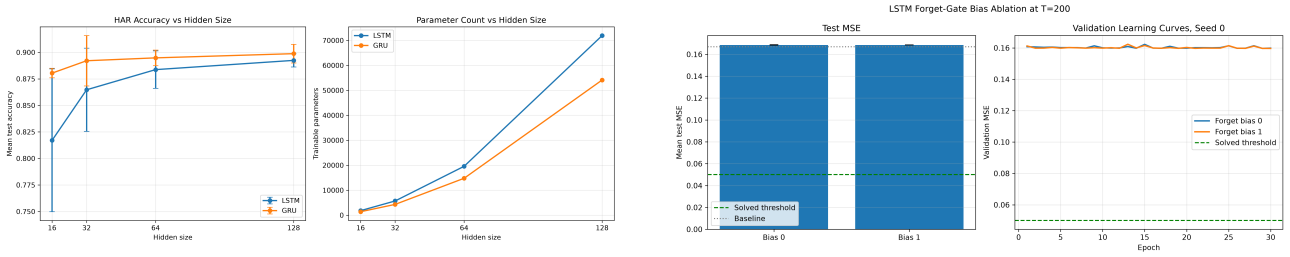


Figure 5. Hidden-state gradient norms across T = 100 (log scale). The notebook reports post-training early-step gradient norms of 2.44 × 10■■ (RNN), 5.76 × 10■¹■ (LSTM) and 3.34 × 10■■ (GRU), showing architecture-dependent gradient propagation.

5. Problem 1 — Capacity and assigned ablation



Figures 6–7. HAR hidden-size sweep and LSTM forget-gate-bias ablation at T = 200.

Increasing hidden size generally raises accuracy while increasing parameter count and epoch time. GRU-32 reaches 89.21% mean accuracy with 4,326 parameters, close to LSTM-128 at 89.24% with 71,942 parameters. GRU-64 reaches 89.48% with 14,790 parameters. These results show a useful compactness/accuracy trade-off.

The final BITS ID digit is odd, so the assigned ablation compares LSTM forget-gate bias 0 vs 1. Mean test MSE is 0.168581 ± 0.000256 (bias 0) and 0.168532 ± 0.000084 (bias 1); neither setting solves the task (0/3). The difference is negligible under this training configuration. For a battery-powered wearable, GRU-32 is a compact candidate; GRU-64 gives a small accuracy increase at more than three times the parameters. Energy consumption itself was not measured.

6. Problem 2 — Transformer and HAR Task A

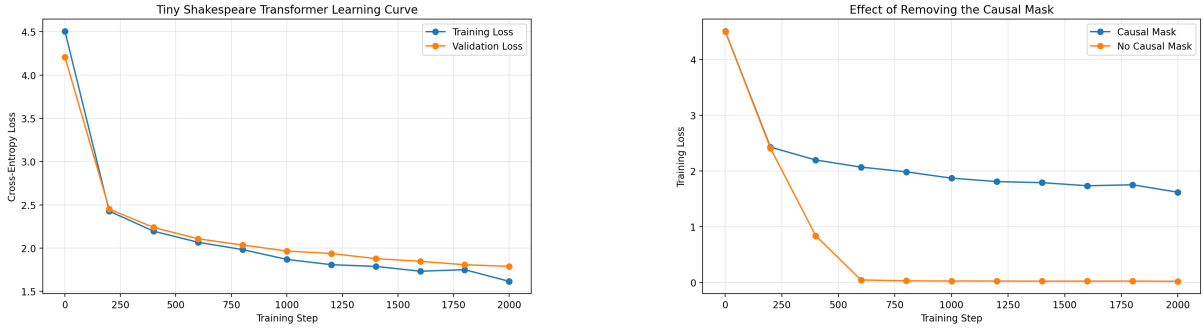
The encoder block parameter count (198,272) matches the hand derivation and notebook. On HAR, learned positional embeddings achieve the highest recorded test accuracy (90.94%), followed by no positional information (90.26%) and sinusoidal encoding (89.38%). Learned positions add 16,512 parameters (129 × 128). The recurrent GRU baseline averages 89.48% over three seeds; note that the Transformer positional ablation has one recorded seed.

Position	Test accuracy	Parameters	s / epoch
learned	90.94%	415,238	1.69
none	90.26%	398,726	1.65
sinusoidal	89.38%	398,726	1.66

Table 3. HAR Transformer positional-information ablation.

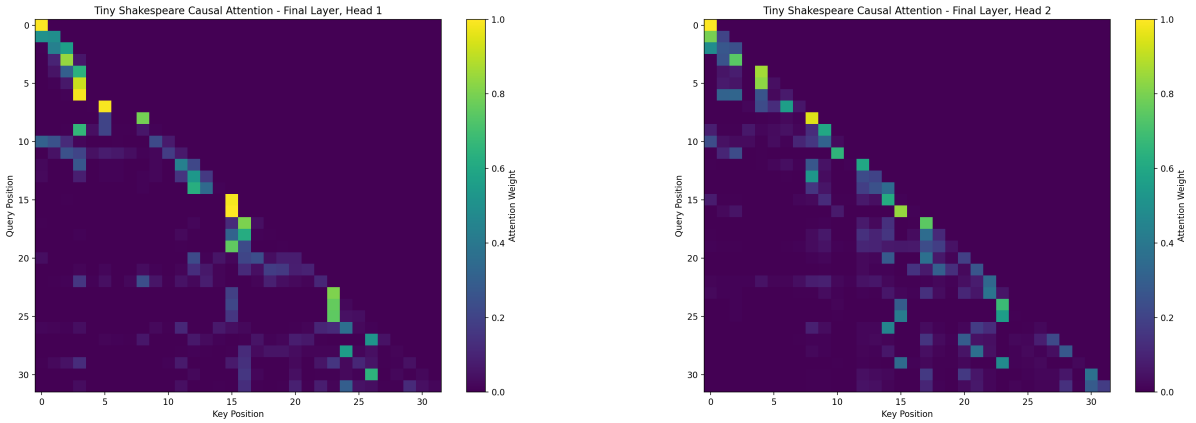
7. Problem 2 — Tiny Shakespeare and ablations

The causal character-level Transformer uses context 128, d_model 128, four heads and four layers; it has 809,793 parameters and was trained for 2,000 steps. The notebook reports validation loss 1.7841 nats, perplexity 5.95 and 2.5739 bits per character (BPC = loss / ln 2). Training time is reported as approximately 71–73 seconds.



Figures 8–9. Causal language-model learning curve and causal-mask loss ablation.

The two final-layer attention heatmaps show different head-specific attention patterns while preserving the lower-triangular causal structure; future-position weights are zero. Head-count ablation at fixed d_model = 128 gives validation losses 1.7876 (1 head), 1.7841 (4 heads) and 1.8086 (8 heads), with 809,793 parameters for each. Four heads give the best recorded validation loss; increasing to eight does not improve performance.



Figures 10–11. Two causal attention heads; the notebook verifies maximum future-position attention = 0.

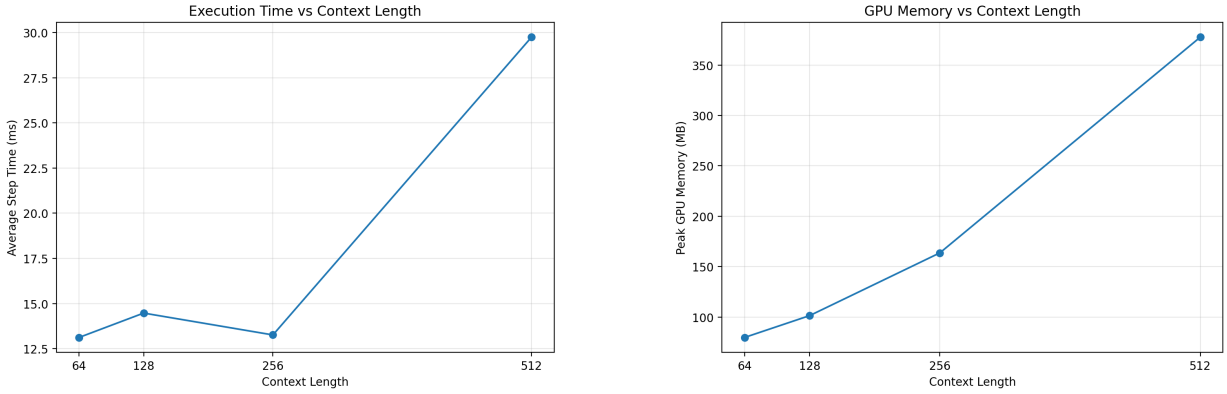
Removing the causal mask reduces validation loss to 0.0191 (BPC 0.0275), versus 1.7841 (2.5739 BPC) for the causal model. This is information leakage, not a genuine improvement: future characters are visible during training. Non-causal generation degenerates into repeated characters, showing why causal masking is required for autoregressive generation.

8. Context-length scaling and conclusions

The notebook's written benchmark reports the following context-length measurements (batch size 8):

Context	Step time (ms)	Peak GPU memory (MB)	Relative n² work
64	13.27	103.53	1×
128	13.53	125.22	4×
256	12.92	187.47	16×
512	28.53	401.35	64×

Table 4. Context-length benchmark as documented in the notebook prose. Attention compute scales approximately as O(n²d).



Figures 12–13. Execution time and peak GPU memory versus context length.

Memory rises with context length, with the 512-token setting the most demanding. Measured step time remains similar through 256 tokens and increases at 512; it does not follow an exact quadratic curve in this benchmark. This is consistent with O(n²d) describing attention's dominant theoretical interaction cost while wall-clock time also reflects framework overhead and hardware effects.

Overall, the repository's experiments favor GRU for the adding task and recurrent HAR comparison, while the learned-position Transformer obtains the strongest single-run HAR accuracy. Four-head causal attention is the best of the tested language-model head counts. The non-causal ablation demonstrates that a very low loss can be invalid when future information leaks into predictions.

AI-use statement and references

AI-use statement: ChatGPT was used to assist with organizing and formatting the report and with concise explanations of deep-learning concepts. The experimental metrics and observations are reported from the completed assignment experiments; the references below are external background sources for the methods and dataset discussed. The student should review this disclosure and ensure it matches the course's AI-use policy.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Background on neural networks, sequence models and optimization.

2. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. https://doi.org/10.1162/NECO.1997.9.8.1735

3. Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP, 1724-1734. https://aclanthology.org/D14-1179/

4. Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS. https://arxiv.org/abs/1706.03762

5. Reyes-Ortiz, J., et al. (2013). Human Activity Recognition Using Smartphones. UCI Machine Learning Repository. https://doi.org/10.24432/C54S4K

Scope note: The cited works provide theoretical and dataset background; they are not presented as sources for the report's experiment-specific numerical results.